

LIBRARY MANAGEMENT SYSTEM

```
import javax.swing.*;

import javax.swing.table.DefaultTableModel;

import java.awt.*;

import java.io.*;

import java.util.ArrayList;

import java.util.List;


/*
 * LIBRARY MANAGEMENT SYSTEM
 * -----
 * A simple Java Swing mini-project.
 * Features: Add, Search, Issue, Return, Delete books.
 * Data is stored in a plain text file (library_data.txt) so it
 * persists between runs — no database setup required.
 *
 * HOW TO RUN:
 * javac LibraryManagementSystem.java
 * java LibraryManagementSystem
 */

// ----- Book class -----

class Book implements Serializable {
```

```
private String id;

private String title;

private String author;

private boolean issued;


public Book(String id, String title, String author, boolean issued) {

    this.id = id;

    this.title = title;

    this.author = author;

    this.issued = issued;
}


public String getId() { return id; }

public String getTitle() { return title; }

public String getAuthor() { return author; }

public boolean isIssued() { return issued; }

public void setIssued(boolean issued) { this.issued = issued; }


// Convert book to a line for saving in the text file
public String toFileString() {

    return id + "|" + title + "|" + author + "|" + issued;
}


// Recreate a book from a saved line
public static Book fromFileString(String line) {

    String[] parts = line.split("\\|");
```

```
        return new Book(parts[0], parts[1], parts[2], Boolean.parseBoolean(parts[3]));
    }
}
```

```
// ----- Library class (handles logic + file storage) -----
```

```
class Library {

    private List<Book> books = new ArrayList<>();

    private final String FILE_NAME = "library_data.txt";

    public Library() {

        loadFromFile();

    }

    public List<Book> getBooks() {

        return books;

    }

    public boolean addBook(String id, String title, String author) {

        for (Book b : books) {

            if (b.getId().equals(id)) return false; // duplicate ID

        }

        books.add(new Book(id, title, author, false));

        saveToFile();

        return true;

    }

}
```

```
public boolean deleteBook(String id) {  
    boolean removed = books.removeIf(b -> b.getId().equals(id));  
    if (removed) saveToFile();  
    return removed;  
}
```

```
public String issueBook(String id) {  
    for (Book b : books) {  
        if (b.getId().equals(id)) {  
            if (b.isIssued()) return "Book is already issued.";  
            b.setIssued(true);  
            saveToFile();  
            return "Book issued successfully.";  
        }  
    }  
    return "Book not found.";  
}
```

```
public String returnBook(String id) {  
    for (Book b : books) {  
        if (b.getId().equals(id)) {  
            if (!b.isIssued()) return "Book was not issued.";  
            b.setIssued(false);  
            saveToFile();  
            return "Book returned successfully.";  
        }  
    }
```

```
    }  
    return "Book not found.";  
}
```

```
public List<Book> search(String keyword) {  
    List<Book> result = new ArrayList<>();  
    keyword = keyword.toLowerCase();  
    for (Book b : books) {  
        if (b.getId().toLowerCase().contains(keyword)  
            || b.getTitle().toLowerCase().contains(keyword)  
            || b.getAuthor().toLowerCase().contains(keyword)) {  
            result.add(b);  
        }  
    }  
    return result;  
}
```

```
private void saveToFile() {  
    try (PrintWriter pw = new PrintWriter(new FileWriter(FILE_NAME))) {  
        for (Book b : books) {  
            pw.println(b.toString());  
        }  
    } catch (IOException e) {  
        System.out.println("Error saving data: " + e.getMessage());  
    }  
}
```

```

private void loadFromFile() {
    File file = new File(FILE_NAME);
    if (!file.exists()) return;
    try (BufferedReader br = new BufferedReader(new FileReader(file))) {
        String line;
        while ((line = br.readLine()) != null) {
            if (!line.trim().isEmpty()) {
                books.add(Book.fromFileString(line));
            }
        }
    } catch (IOException e) {
        System.out.println("Error loading data: " + e.getMessage());
    }
}
}

```

// ----- GUI class -----

```

public class LibraryManagementSystem extends JFrame {

    private Library library = new Library();
    private DefaultTableModel tableModel;
    private JTable table;
    private JTextField idField, titleField, authorField, searchField;

    public LibraryManagementSystem() {

```

```
setTitle("Library Management System");

setSize(800, 500);

setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

setLocationRelativeTo(null);

setLayout(new BorderLayout(10, 10));


add(buildFormPanel(), BorderLayout.NORTH);
add(buildTablePanel(), BorderLayout.CENTER);
add(buildButtonPanel(), BorderLayout.SOUTH);


refreshTable(library.getBooks());
}


private JPanel buildFormPanel() {
    JPanel panel = new JPanel(new GridLayout(2, 5, 5, 5));
    panel.setBorder(BorderFactory.createTitledBorder("Book Details"));


    idField = new JTextField();
    titleField = new JTextField();
    authorField = new JTextField();
    searchField = new JTextField();


    panel.add(new JLabel("Book ID:"));
    panel.add(idField);
    panel.add(new JLabel("Title:"));
    panel.add(titleField);
```

```

panel.add(new JLabel("Author:"));

panel.add(authorField);
panel.add(new JLabel("Search (ID/Title/Author):"));
panel.add(searchField);

JButton searchBtn = new JButton("Search");
JButton resetBtn = new JButton("Show All");
panel.add(searchBtn);
panel.add(resetBtn);

searchBtn.addActionListener(e ->
refreshTable(library.search(searchField.getText().trim())));

resetBtn.addActionListener(e -> {
    searchField.setText("");
    refreshTable(library.getBooks());
});

return panel;
}

private JScrollPane buildTablePanel() {
    tableModel = new DefaultTableModel(new String[]{"ID", "Title", "Author", "Status"}, 0) {
        public boolean isCellEditable(int row, int col) { return false; }
    };

    table = new JTable(tableModel);

    table.getSelectionModel().addListSelectionListener(e -> {

```

```
int row = table.getSelectedRow();  
if (row >= 0) {  
    idField.setText(tableModel.getValueAt(row, 0).toString());  
    titleField.setText(tableModel.getValueAt(row, 1).toString());  
    authorField.setText(tableModel.getValueAt(row, 2).toString());  
}  
});  
return new JScrollPane(table);  
}
```

```
private JPanel buildButtonPanel() {  
    JPanel panel = new JPanel(new FlowLayout());  
  
    JButton addBtn = new JButton("Add Book");  
    JButton issueBtn = new JButton("Issue Book");  
    JButton returnBtn = new JButton("Return Book");  
    JButton deleteBtn = new JButton("Delete Book");  
    JButton clearBtn = new JButton("Clear Fields");  
  
    panel.add(addBtn);  
    panel.add(issueBtn);  
    panel.add(returnBtn);  
    panel.add(deleteBtn);  
    panel.add(clearBtn);  
  
    addBtn.addActionListener(e -> {
```

```
String id = idField.getText().trim();
String title = titleField.getText().trim();
String author = authorField.getText().trim();
if (id.isEmpty() || title.isEmpty() || author.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Please fill all fields.");
    return;
}
boolean added = library.addBook(id, title, author);
if (added) {
    refreshTable(library.getBooks());
    clearFields();
} else {
    JOptionPane.showMessageDialog(this, "A book with this ID already exists.");
}
});
```

```
issueBtn.addActionListener(e -> {
    String msg = library.issueBook(idField.getText().trim());
    JOptionPane.showMessageDialog(this, msg);
    refreshTable(library.getBooks());
});
```

```
returnBtn.addActionListener(e -> {
    String msg = library.returnBook(idField.getText().trim());
    JOptionPane.showMessageDialog(this, msg);
    refreshTable(library.getBooks());
});
```

```
});
```

```
deleteBtn.addActionListener(e -> {  
    boolean deleted = library.deleteBook(idField.getText().trim());  
    if (deleted) {  
        refreshTable(library.getBooks());  
        clearFields();  
    } else {  
        JOptionPane.showMessageDialog(this, "Book not found.");  
    }  
});
```

```
clearBtn.addActionListener(e -> clearFields());
```

```
return panel;
```

```
}
```

```
private void refreshTable(List<Book> books) {  
    tableModel.setRowCount(0);  
    for (Book b : books) {  
        tableModel.addRow(new Object[]{  
            b.getId(), b.getTitle(), b.getAuthor(),  
            b.isIssued() ? "Issued" : "Available"  
        });  
    }  
}
```

```
private void clearFields() {  
    idField.setText("");  
    titleField.setText("");  
    authorField.setText("");  
}  
  
public static void main(String[] args) {  
    SwingUtilities.invokeLater(() -> new LibraryManagementSystem().setVisible(true));  
}  
}
```